

```
import nltk
```

```
nltk.download('punkt')
```

```
nltk.download('punkt_tab')
```

```
[nltk_data] Downloading package punkt to /root/nltk_data...
```

```
[nltk_data]   Unzipping tokenizers/punkt.zip.
```

```
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
```

```
[nltk_data]   Unzipping tokenizers/punkt_tab.zip.
```

```
True
```

```
streamlit langchain faiss-cpu pypdf google-generativeai sentence-transformers
```

```
Requirement already satisfied: pyasn1<0.7.0,>=0.6.1 in /usr/local/lib/python3.12/dist-packages (0.6.1)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (1.17.0)
Requirement already satisfied: mpmath<1.4,>=1.1.0 in /usr/local/lib/python3.12/dist-packages (1.3.0)
Requirement already satisfied: sniffio>=1.1 in /usr/local/lib/python3.12/dist-packages (1.3.1)
Downloading streamlit-1.51.0-py3-none-any.whl (10.2 MB)
 10.2/10.2 MB 111.6 MB/s eta 0:00
Downloading faiss_cpu-1.12.0-cp312-cp312-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl (31.4 MB)
 31.4/31.4 MB 62.2 MB/s eta 0:00
Downloading pypdf-6.1.3-py3-none-any.whl (323 kB)
 323.9/323.9 kB 28.1 MB/s eta 0:00
Downloading pydeck-0.9.1-py2.py3-none-any.whl (6.9 MB)
 6.9/6.9 MB 119.7 MB/s eta 0:00
Installing collected packages: pypdf, faiss-cpu, pydeck, streamlit
Successfully installed faiss-cpu-1.12.0 pydeck-0.9.1 pypdf-6.1.3 streamlit-1.51.0
```

```
# Install required packages

!pip install -q google-genai sentence-transformers faiss-cpu pypdf nltk

#Imports

import os

import numpy as np

from typing import List, Dict, Any

import nltk

nltk.download('punkt')

from sentence_transformers import SentenceTransformer

import faiss

from pypdf import PdfReader
```

```
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data] Package punkt is already up-to-date!
```

```
# 1) Configure Gemini
# -----
GEMINI_API_KEY = os.environ.get("GOOGLE_API_KEY")
if not GEMINI_API_KEY:
    GEMINI_API_KEY = input("Enter your Gemini API key: ").strip()
os.environ["GOOGLE_API_KEY"] = GEMINI_API_KEY
from google import genai
client = genai.Client(api_key=GEMINI_API_KEY)
GEMINI_MODEL = "gemini-2.5-flash" # Adjust to your available model
```

```
Enter your Gemini API key: AIzaSyDvmXzyjX0HipKFddCzIUxpCWBFIb68fH0
```

```

#2) Upload PDF and extract text
# -----
from google.colab import files
print("Upload your PDF file:")
uploaded = files.upload()
if not uploaded:
    raise SystemExit("No PDF uploaded.")
pdf_file = list(uploaded.keys())[0]
reader = PdfReader(pdf_file)
raw_text = ""
for page in reader.pages:
    page_text = page.extract_text()
    if page_text:
        raw_text += page_text + "\n"
print(f"PDF loaded: {pdf_file}, {len(raw_text)} characters extracted.")

```

Upload your PDF file:

RAG Questions.pdf

RAG Questions.pdf(application/pdf) - 2055083 bytes, last modified: 22/10/2025 - 100% done

Saving RAG Questions.pdf to RAG Questions.pdf

PDF loaded: RAG Questions.pdf, 102718 characters extracted.

```

# 3) Chunking function
# -----
def chunk_text(text: str, max_words: int = 200, overlap_words: int = 40) ->
    sentences = nltk.tokenize.sent_tokenize(text)
    chunks = []
    current = []
    current_len = 0
    idx = 0
    for sent in sentences:
        words = sent.split()
        if current_len + len(words) <= max_words:
            current.append(sent)
            current_len += len(words)
        else:
            chunk_text_str = " ".join(current).strip()
            if chunk_text_str:
                chunks.append({"id": idx, "text": chunk_text_str})
                idx += 1
            # create overlapping window
            window = []
            wcount = 0
            for s in reversed(current):
                sw = s.split()
                if wcount + len(sw) > overlap_words:
                    break
                window.insert(0, s)
                wcount += len(sw)
            current = window + [sent]
            current_len = sum(len(s.split()) for s in current)
    if current:
        chunks.append({"id": idx, "text": " ".join(current).strip()})
    return chunks

```

```
chunks = chunk_text(raw_text, max_words=220, overlap_words=50)
print(f"Created {len(chunks)} chunks.")
```

Created 68 chunks.

#4) Create embeddings

```
# -----
embed_model = SentenceTransformer("all-MiniLM-L6-v2")
texts = [c["text"] for c in chunks]
embs = embed_model.encode(texts, show_progress_bar=True, convert_to_numpy=True)
emb_dim = embs.shape[1]
```

/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94: Us
The secret `HF\_TOKEN` does not exist in your Colab secrets.

To authenticate with the Hugging Face Hub, create a token in your settings ta
You will be able to reuse this secret in all of your notebooks.

Please note that authentication is recommended but still optional to access p

```
warnings.warn(
modules.json: 100% 349/349 [00:00<00:00, 29.7kB/s]
config_sentence_transformers.json: 100% 116/116 [00:00<00:00, 12.4kB/s]
README.md: 10.5k/? [00:00<00:00, 825kB/s]
sentence_bert_config.json: 100% 53.0/53.0 [00:00<00:00, 4.95kB/s]
config.json: 100% 612/612 [00:00<00:00, 57.5kB/s]
model.safetensors: 100% 90.9M/90.9M [00:00<00:00, 152MB/s]
tokenizer_config.json: 100% 350/350 [00:00<00:00, 25.1kB/s]
vocab.txt: 232k/? [00:00<00:00, 12.0MB/s]
tokenizer.json: 466k/? [00:00<00:00, 18.5MB/s]
special_tokens_map.json: 100% 112/112 [00:00<00:00, 9.43kB/s]
config.json: 100% 190/190 [00:00<00:00, 10.2kB/s]
Batches: 100% 3/3 [00:08<00:00, 2.47s/it]
```

5) Build FAISS index

```
# -----
faiss.normalize_L2(embs)
index = faiss.IndexFlatIP(emb_dim)
index.add(embs)
print("FAISS index built with", index.ntotal, "vectors.")
```

FAISS index built with 68 vectors.

6) Retrieval function

```
# -----
def retrieve(query: str, top_k: int = 5) -> List[Dict[str, Any]]:
```

```

q_emb = embed_model.encode([query], convert_to_numpy=True)
faiss.normalize_L2(q_emb)
D, I = index.search(q_emb, top_k)
results = []
for idx, score in zip(I[0], D[0]):
    if idx < 0:
        continue
    results.append({"id": int(idx), "score": float(score), "text": chunk})
return results

```

```

# 7) LLM query using updated Gemini API
# -----
def call_gemini(prompt: str) -> str:
    response = client.models.generate_content(
        model=GEMINI_MODEL,
        contents=prompt
    )
    return response.text.strip()

def answer_query_with_context(query: str, top_k: int = 5) -> Dict[str, Any]:
    retrieved = retrieve(query, top_k=top_k)
    context_texts = [f"[doc_id:{r['id']}] | score:{r['score']:.4f}]\n{r['text']}" for r in retrieved]
    prompt = f"""
You are a helpful assistant. Use only the context below to answer the question.
CONTEXT:
{"".join(context_texts)}
QUESTION:
{query}
Answer concisely and cite doc_id(s) used in the answer.
"""
    answer = call_gemini(prompt)
    return {"query": query, "answer": answer, "retrieved": retrieved, "prompt": prompt}

```

```

# 8) Example usage
# -----
query = "Explain the steps in the indexing process in a RAG pipeline."
res = answer_query_with_context(query, top_k=3)
print("\nQuery:", query)
print("LLM Answer:", res["answer"])
print("Retrieved docs:", [(r["id"], r["score"]) for r in res["retrieved"]])

```

Query: Explain the steps in the indexing process in a RAG pipeline.

LLM Answer: The indexing process in a RAG pipeline involves four steps: parsing, chunking, encoding, and storing.

- Parsing:** Extracts the document content.
- Chunking:** Splits the extracted content into smaller pieces called chunks.
- Encoding:** Uses an embedding model to convert chunks into dense numerical representations.
- Storing:** Saves these embeddings in a vector database for efficient search.

(doc_id: 5)

Retrieved docs: [(5, 0.5482720136642456), (6, 0.5003253817558289), (10, 0.481...)]

